INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE OCCIDENTE

Tecnologías de desarrollo en el servidor



Entrega 3: Swagger, correos y logging (documento v4)

Presentan
Emilio Daniel Guzmán Seda
Daniel Sebastián Macías Macías
Profesor: Francisco Javier Sevilla Medina

Fecha: martes 10 de noviembre de 2026

Proyecto Integrador — Recall

Documento del proyecto · versión 4 · Entrega 3: Swagger, correos y logging · 10 de noviembre de 2026

Recall es el REST API de una plataforma web que recibe audio de reuniones, clases y entrevistas, lo transcribe y genera resúmenes con IA, sin depender de un dispositivo dedicado. El MVP se mantiene lo más simple posible: un solo API en TypeScript con Express, PostgreSQL en Docker, autenticación propia con JWT, login opcional con Google, correos con nodemailer, logs con pino, documentación interactiva con Swagger y servicios de AWS (S3, Transcribe y Bedrock) para las siguientes entregas.

Cambios respecto a la versión 3

Tema	Versión 3	Versión 4 (esta entrega)
Correos	No había	Servicio único con nodemailer; en desarrollo los correos se ven en Mailpit. Tres correos: verificación, recuperación de contraseña y «Tu resumen está listo»
Verificación de correo	No había	`emailVerified` en User; subir y procesar audio exige el correo verificado (403 `EMAIL_NOT_VERIFIED`)
Recuperar contraseña	Fuera del alcance	`forgot-password` y `reset-password` con un enlace de un solo uso que vence en 1 hora
Tokens de un solo uso	No había	Tabla `AuthToken`: solo se guarda el sha256 del token, vence y sirve una sola vez
Logging	`console.log`	pino: una línea por petición con su id (`X-Request-Id`), niveles por `LOG_LEVEL`, JSON en producción y redacción de secretos
Documentación del API	Solo el README	Swagger (OpenAPI 3) en `/docs` con Authorize, generado desde comentarios `@openapi` junto a cada ruta
Colección de Postman	95 peticiones	129 peticiones: carpetas de verificación de correo, recuperar contraseña y correo del proyecto, que leen los correos desde Mailpit
Seed	6 usuarios, todos verificados	7 usuarios: se agrega Laura Sánchez con el correo sin verificar, para probar el 403
Base de datos	4 tablas, 5 migraciones	5 tablas (se agrega `AuthToken`), 6 migraciones

Datos generales

Campo	Valor
Nombre del proyecto	Recall
Tipo	REST API con TypeScript y Express, con vistas en Handlebars
Repositorio	github.com/sebastian0023/Recall (privado)
Acceso del profesor	fsevilla@gmail.com como colaborador
Materia	Tecnologías de desarrollo en el servidor · ITESO

Integrantes

Nombre	Responsabilidades
Daniel Sebastián Macías Macías	Conexión a la BD, modelos y migraciones, CRUD con Prisma, validaciones con zod, manejo de errores, autenticación con JWT y bcrypt, login con Google. En la entrega 3: logging con pino, Swagger, servicio de correo, tokens de un solo uso, verificación, recuperación de contraseña y el correo «Tu resumen está listo»
Emilio Daniel Guzmán Seda	Seed, vistas con Handlebars (incluido el registro), roles y permisos, colección de Postman y casos de seguridad, CI/CD (ESLint, SonarQube, Docker) y documento. En la entrega 3: pruebas de los flujos de correo y de los logs, corrección de `/docs` en Windows, revisión de `.env.example` y documento v4

Ambos integrantes trabajan en ramas por funcionalidad que llegan a `main` mediante Pull Request.

Descripción

Problema

En reuniones, clases y entrevistas se pierde información porque tomar notas y prestar atención al mismo tiempo es difícil. Las herramientas que lo resuelven suelen requerir hardware de pago, están pensadas para el inglés o cobran suscripciones en dólares.

Solución propuesta

Recall permite subir un archivo de audio. El API lo guarda en Amazon S3, lo transcribe con Amazon Transcribe y genera un resumen con Amazon Bedrock. El usuario consulta y edita sus grabaciones, transcripciones y resúmenes desde el API o desde las vistas web.

Alcance del MVP

- Registro e inicio de sesión con correo y contraseña, o con Google.
- Roles `USER` y `ADMIN`: cada usuario administra solo sus datos; el administrador ve todo.

- CRUD de grabaciones; el archivo de audio se guarda en S3.
- Transcripción en español con Amazon Transcribe.
- Resumen con Amazon Bedrock usando una de tres plantillas fijas: reunión, clase o entrevista.
- Consulta y edición de transcripciones y resúmenes, desde el API y desde vistas web.

Esta entrega (Swagger, correos y logging)

- Un solo módulo de correo (`src/services/mail.service.ts`) configurado con variables de entorno; en desarrollo los correos se ven en Mailpit.
- Verificación de correo al registrarse, con reenvío y una regla: subir y procesar audio exige el correo verificado.
- Recuperación de contraseña con un enlace de un solo uso, sin revelar qué correos existen.
- Un correo propio del dominio: «Tu resumen está listo», al dueño de la grabación.
- Logs con pino: una línea por petición, id de petición, niveles, errores con su stack y sin contraseñas ni tokens.
- Documentación interactiva de todo el API con Swagger en `/docs`.

Fuera del alcance

- Carpetas, etiquetas y plantillas personalizadas.
- Transcripción en tiempo real del audio, aplicación móvil y compartir grabaciones.
- Colas para enviar correos en segundo plano y reintentos automáticos.
- Enviar los logs a un servicio externo o guardarlos en archivos: por ahora salen por la salida estándar.
- Despliegue en la nube: el proyecto corre en local (la imagen Docker queda lista para un despliegue futuro).

Proyectos similares y referencias

Recall toma como referencia principal a Plaud, pero elimina la dependencia de hardware y se enfoca en usuarios hispanohablantes.

Referencia	Qué ofrece	Diferencia con Recall
Plaud	Grabadoras físicas (Note, Note Pro, NotePin) de USD 159 a 189, con app que transcribe y resume con plantillas. Plan Pro de USD 17.99 al mes para 1,200 minutos.	Recall no requiere comprar un dispositivo: graba desde el navegador.
Otter.ai	Transcripción de reuniones y audios importados en 6 idiomas, con resúmenes y bot para Zoom, Teams y Meet. Plan gratuito limitado a 3 archivos importados.	Recall prioriza archivos subidos y grabación presencial, sin límite artificial de importaciones.

Referencia	Qué ofrece	Diferencia con Recall
Fireflies.ai, tl;dv	Bots que se unen a videollamadas, transcriben y generan notas.	Enfocados en reuniones en línea; Recall cubre clases y entrevistas presenciales.
Grabadora de Google Pixel	Transcripción local en el teléfono.	Exclusiva de un modelo de teléfono y sin resúmenes por plantilla.

Motivo para desarrollar una herramienta similar

- **Sin hardware:** cualquier persona con navegador puede usarla, sin pagar un dispositivo de más de USD 150.
- **Español latino como prioridad:** plantillas y resúmenes pensados para contextos académicos y laborales en México.
- **Control de los datos:** el audio vive en un bucket propio, con políticas de retención definidas por el proyecto.
- **Fin académico:** integra autenticación, base de datos relacional, almacenamiento en la nube, contenedores y patrones de diseño.

Stack de tecnologías

Backend y base de datos

Capa	Tecnología	Uso en el proyecto
Lenguaje	TypeScript (strict)	Tipado estático
Runtime	Node.js 24 LTS	Ejecución del API
Framework HTTP	Express 5	Rutas, middlewares, controladores y vistas
Base de datos	PostgreSQL 17	Usuarios, grabaciones, transcripciones y resúmenes
ORM y migraciones	Prisma 7	Esquema, migraciones versionadas y consultas tipadas
Validación	zod	Esquemas de body, params y query
Autenticación	bcryptjs y jsonwebtoken	Hash de contraseñas y sesión con JWT (HS256)
Login con proveedor	Google OAuth 2.0 / OpenID Connect	Login opcional con una cuenta de Google
Vistas	Handlebars (express-handlebars)	Registro, login, listados y detalle con datos de la BD
Correo	nodemailer + Mailpit (Docker)	Envío por SMTP; Mailpit recibe los correos en desarrollo y los muestra en http://localhost:8025
Logging	pino + pino-http	Una línea por petición con su id, niveles y redacción de secretos

Capa	Tecnología	Uso en el proyecto
Documentación	swagger-jsdoc + swagger-ui-express	Especificación OpenAPI 3 y Swagger UI en `/docs`
Contenedores	Docker Compose / Dockerfile	PostgreSQL local e imagen del API
Calidad y CI/CD	ESLint, SonarQube Cloud, GitHub Actions	Lint, quality gate, pruebas de la colección e imagen Docker en cada push

Librerías

Librería	Para qué se usa	Entrega
express, cors, dotenv	Servidor HTTP, orígenes permitidos y variables de entorno	Planeación
prisma, @prisma/client, @prisma/adapter-pg, pg	Esquema, migraciones y conexión a PostgreSQL	Planeación / Entrega 1
zod	Validación de body, params y query	Entrega 1
express-handlebars	Motor de vistas	Entrega 1
bcryptjs	Hash de contraseñas (10 rondas)	Entrega 2
jsonwebtoken	Firma y verificación de los JWT	Entrega 2
cookie-parser	Leer la cookie de sesión de las vistas	Entrega 2
nodemailer	Enviar los correos por SMTP	Entrega 3 (esta)
pino, pino-http, pino-pretty	Logger único, línea por petición y formato legible en desarrollo	Entrega 3 (esta)
swagger-jsdoc, swagger-ui-express	Generar la especificación OpenAPI desde comentarios `@openapi` y servirla en `/docs`	Entrega 3 (esta)
eslint, typescript-eslint	Linter de TypeScript (también corre en CI)	Entrega 1
tsx, typescript	Ejecución en desarrollo, seed y compilación	Planeación
multer, @aws-sdk/client-s3	Recibir el audio y subirlo a S3	Siguiente entrega
@aws-sdk/client-transcribe, client-bedrock-runtime	Transcribir y resumir	Siguientes entregas

El login con Google no usa una librería: el API habla directamente con los endpoints de autorización y de token de Google con `fetch`.

Servicios externos

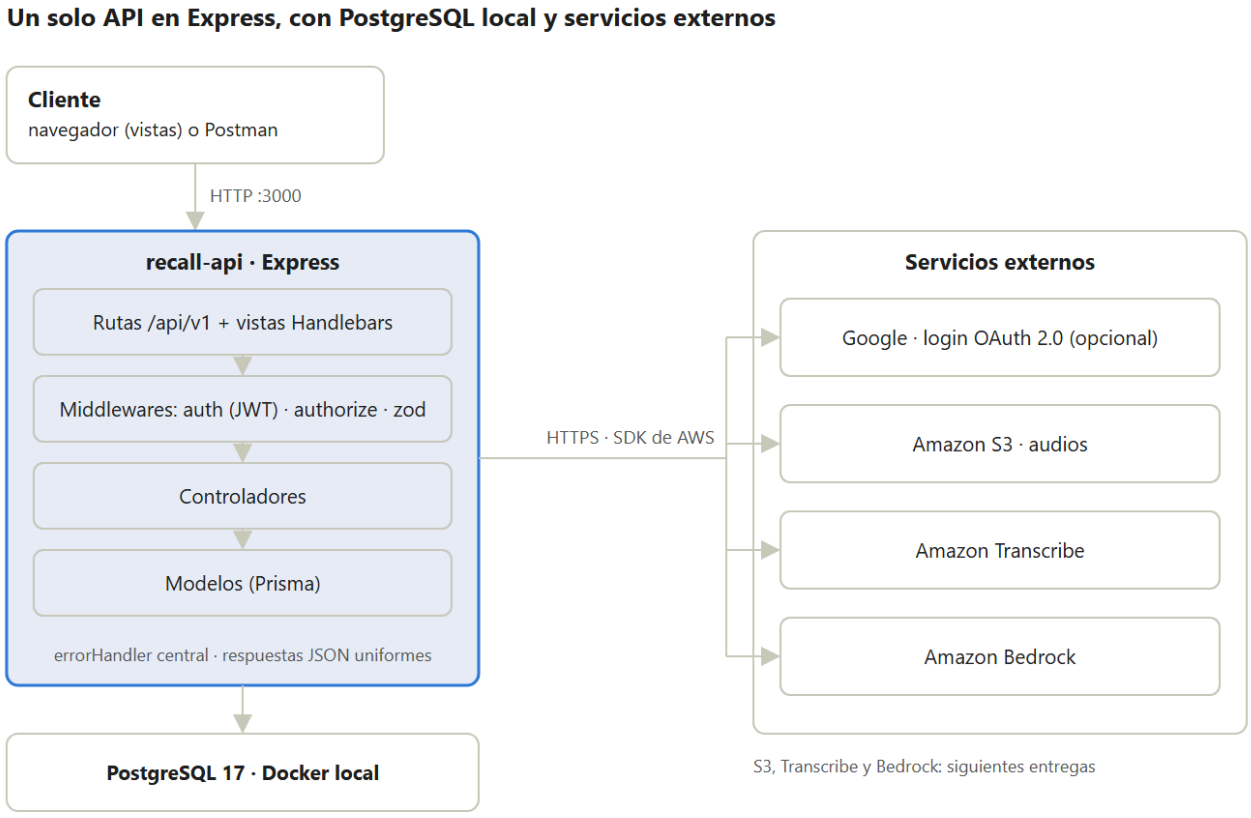
Servicio	Uso
Google (OAuth 2.0 / OpenID Connect)	Login opcional con una cuenta de Google. Se activa con tres variables de entorno
Amazon S3	Guardar los archivos de audio en un bucket privado (siguiente entrega)
Amazon Transcribe	Transcripción por lotes en español (es-US o es-MX) con separación de hablantes
Amazon Bedrock	Generar el resumen con un modelo de lenguaje a partir de la plantilla elegida

Alternativas investigadas

Necesidad	Elegida	Alternativas
Framework	Express	NestJS, Fastify
ORM	Prisma	TypeORM, Drizzle
Validación	zod	express-validator, joi
Vistas	Handlebars	EJS, Pug, cliente React separado
Autenticación	JWT propio con bcrypt	Amazon Cognito, Auth0, sesiones en servidor
Login con proveedor	Google con OAuth 2.0 directo	Passport.js, Cognito Hosted UI
Transcripción	Amazon Transcribe	OpenAI Whisper, Deepgram, AssemblyAI
Resúmenes	Amazon Bedrock	OpenAI API, Ollama local
Almacenamiento	Amazon S3	Cloudflare R2, MinIO
Correo	nodemailer + Mailpit	Resend, SendGrid, Amazon SES, Ethereal, Mailtrap
Logging	pino	winston, log4js, morgan

En la versión 2 se usaba Amazon Cognito. Se cambió a autenticación propia porque la entrega pide implementar el registro, el login y el manejo de tokens en el API, y porque Cognito dependía de una sola cuenta de AWS para probar cualquier cosa. Con JWT propio, cualquier integrante (y el profesor) puede correr y probar todo en local.

Arquitectura



Arquitectura del MVP · un API en Express, PostgreSQL local y servicios externos

El API corre en la máquina del desarrollador y se conecta a PostgreSQL en Docker. Sigue el patrón MVC:

- **Rutas** (`src/routes/`): definen los endpoints de cada módulo bajo `/api/v1` y las rutas de las vistas, y aplican los middlewares.
- **Middlewares** (`src/middlewares/`): `requestLogger` (línea por petición e id), `requireAuth` y `viewAuth` (verifican el JWT), `requireRole`, `requireSelfOrAdmin` y `requireVerifiedEmail` (permisos), `validate` (zod), `notFound` (404) y `errorHandler` (respuesta de error uniforme y log del error).
- **Servicios** (`src/services/`): el módulo de correo y sus plantillas.
- **Controladores** (`src/controllers/`): reciben la petición validada, revisan permisos, consultan la BD con Prisma y responden JSON o renderizan una vista.
- **Modelos** (`prisma/schema.prisma` y `src/models/`): esquema de la BD y tipos de entrada de cada entidad.
- **Vistas** (`views/`): plantillas Handlebars con un layout base y navegación.

Flujo de procesamiento (siguientes entregas)

- `POST /recordings` recibe el audio, lo sube a S3 y crea la grabación con estado UPLOADED.
- `POST /recordings/:id/process` inicia un trabajo en Transcribe y cambia el estado a PROCESSING (hoy solo cambia el estado).
- Cuando el trabajo termina, el API guarda la transcripción, pide el resumen a Bedrock, lo guarda y cambia el estado a DONE (o FAILED si algo falla).

Base de datos

La base de datos es relacional (PostgreSQL 17) con 5 tablas definidas en `prisma/schema.prisma`. Las llaves primarias son UUID y las 6 migraciones están versionadas en `prisma/migrations`. El diagrama se actualizó: User tiene `emailVerified` y la tabla nueva `AuthToken` guarda los enlaces de un solo uso.

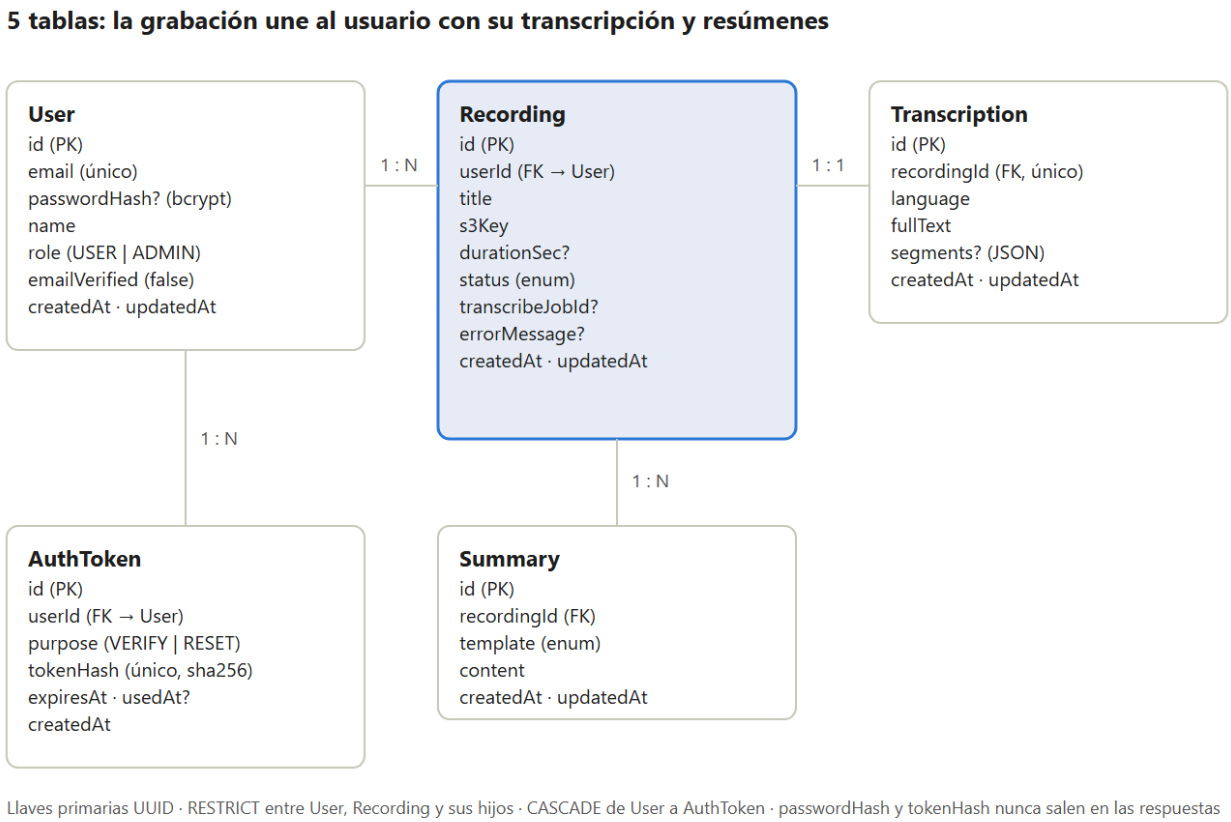


Diagrama entidad-relación · 5 tablas en PostgreSQL

Modelo User

Campo	Tipo	Notas
`id`	UUID	Llave primaria
`email`	texto, único	Duplicado responde 409
`passwordHash`	texto, opcional	Hash bcrypt de 10 rondas. Nulo para cuentas creadas con Google. Prisma lo omite en todas las consultas, así que nunca llega a una respuesta
`name`	texto	Hasta 100 caracteres
`role`	enum `USER` · `ADMIN`	`USER` por defecto. Nunca se toma del registro ni de Google
`emailVerified`	booleano	`false` por defecto. Pasa a `true` al abrir el enlace de verificación, al restablecer la contraseña o al entrar con Google
`createdAt`, `updatedAt`	fecha	Automáticos

Modelo AuthToken

Campo	Tipo	Notas
`id`	UUID	Llave primaria
`userId`	UUID	Dueño del token. Si se borra el usuario, se borran sus tokens (CASCADE)
`purpose`	enum `VERIFY_EMAIL` · `RESET_PASSWORD`	Para qué sirve el token
`tokenHash`	texto, único	sha256 del token. El token en claro solo viaja en el correo
`expiresAt`	fecha	Vencimiento (`VERIFY_TOKEN_TTL` o `RESET_TOKEN_TTL`)
`usedAt`	fecha, opcional	Se llena al usarlo; un token con `usedAt` ya no sirve

Relaciones

Relación	Cardinalidad	Explicación
User → Recording	1 : N	Un usuario tiene muchas grabaciones; cada grabación pertenece a un solo usuario (`userId`).
Recording → Transcription	1 : 1	Una grabación tiene a lo más una transcripción; `recordingId` es único.
Recording → Summary	1 : N	Una grabación puede tener varios resúmenes, por ejemplo uno por plantilla.

Relación	Cardinalidad	Explicación
User → AuthToken	1 : N	Un usuario puede tener tokens de verificación y de recuperación; solo uno vigente por propósito.

Índices y reglas

- Índices únicos: `User.email`, `Transcription.recordingId` y `AuthToken.tokenHash`. Un duplicado responde 409.
- Índices de búsqueda: `Recording.userId`, `Summary.recordingId` y `AuthToken(userId, purpose)`.
- `Recording.status` es un enum: UPLOADED, PROCESSING, DONE, FAILED. `Summary.template` es un enum: MEETING, CLASS, INTERVIEW. `User.role` es un enum: USER, ADMIN.
- Los fragmentos por hablante se guardan en un campo JSON (`segments`) para no crear una tabla extra.
- El audio no se guarda en la base de datos, solo su llave en S3 (`s3Key`).

Política de borrado

Borrado físico resuelto en el código. Las llaves foráneas son RESTRICT, así que la base de datos nunca borra en cascada por su cuenta:

Se elimina	Qué pasa
Recording	Se borran sus resúmenes y su transcripción y después la grabación, en una transacción. Responde 204.
User	Si tiene grabaciones no se elimina: responde 409. Primero hay que borrar sus grabaciones.
Transcription / Summary	Se eliminan directamente (no tienen dependientes). Responde 204.

Autenticación

Registro y login con contraseña

- `POST /api/v1/auth/register` recibe `name`, `email` y `password` (8 a 72 caracteres). El esquema es estricto: cualquier otro campo, como `role`, responde 400. El usuario siempre se crea con rol `USER`.
- La contraseña se guarda con **bcrypt** (10 rondas). Nunca se guarda en texto plano y nunca aparece en una respuesta.
- `POST /api/v1/auth/login` devuelve `{ accessToken, tokenType: "Bearer", expiresIn, user }`.

- **Mismo mensaje en el login:** si el correo no existe o la contraseña es incorrecta, responde lo mismo, `401 Incorrect email or password`. Además compara siempre contra un hash bcrypt, aunque el correo no exista, para que el tiempo de respuesta tampoco revele qué correos están registrados.

Sesión con JWT

Aspecto	Cómo se hace
Algoritmo	HS256, firmado con `JWT_SECRET` (mínimo 32 caracteres). Solo se acepta HS256, así que un token con `alg: none` se rechaza
Contenido	Solo `sub` (id del usuario), `iat` y `exp`. El rol no va en el token: se lee de la BD en cada petición, así un cambio de rol surte efecto de inmediato
Vigencia	`JWT_EXPIRES_IN` (por defecto `1h`). Un token vencido responde `401 Token expired`
API	`Authorization: Bearer <token>`. Sin token responde `401 Unauthorized`; un token alterado o firmado con otra clave, `401 Invalid token`
Vistas	El mismo JWT en la cookie `recall_session`, con `httpOnly`, `sameSite=lax` y `secure` en producción. Las rutas del API no aceptan la cookie, solo el header, para evitar ataques CSRF
Producción	El API se niega a arrancar con el `JWT_SECRET` de ejemplo cuando `NODE_ENV=production`

Login con Google

Flujo OAuth 2.0 / OpenID Connect con código de autorización, implementado sin librerías externas:

1. `GET /auth/google` genera un `state` y un `nonce` aleatorios, los guarda en una cookie `httpOnly` de 10 minutos y redirige a Google.
2. Google regresa a la URL de `GOOGLE_CALLBACK_URL` (por defecto `/google_confirm`). El API revisa que el `state` coincida con el de la cookie, cambia el código por el `id_token` y valida el emisor, la audiencia, la vigencia, el `nonce` y que el correo esté **verificado** por Google.
3. Si ya existe un usuario con ese correo, entra a esa cuenta; si no, la crea sin contraseña y con rol `USER`. El rol nunca viene de Google.
4. Abre la misma sesión que el login con contraseña (JWT en la cookie), así que los permisos son idénticos.

Es opcional: sin `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET` y `GOOGLE_CALLBACK_URL` el botón «Continuar con Google» no aparece. Las credenciales solo viven en el `.env`.

Correos

El API se comunica con los usuarios por correo para tres cosas: confirmar que el correo es suyo, recuperar la contraseña sin ayuda de un administrador y avisar cuando su resumen está listo.

Cómo se envían

- **Un solo módulo:** todo el envío pasa por `src/services/mail.service.ts`, el único lugar que crea el transporte de nodemailer. Expone una función por correo: `sendVerificationEmail`, `sendPasswordResetEmail` y `sendSummaryReadyEmail`. Los controladores no arman HTML ni crean transportes.
- **Plantillas en un solo lugar:** `src/services/mail.templates.ts` tiene el layout común y el contenido de cada correo. Cada correo sale con **versión HTML y versión de texto**, un asunto claro y el nombre del proyecto.
- **Configuración por variables de entorno:** `SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS` y `MAIL_FROM`. Aparecen en `.env.example` sin valores reales; ninguna credencial está en el repositorio.
- **Enlaces con `APP_URL`:** `src/lib/appLink.ts` arma los enlaces de los correos. Ningún dominio está escrito en el código, así que funcionan igual en local y en producción.
- **Si el correo falla, la petición no truena:** el envío nunca lanza un error. La petición responde igual (por ejemplo, el registro crea al usuario) y el log registra un `warn Email not sent` con el tipo de correo, el usuario y el motivo. El usuario puede pedir que se le reenvíe el enlace.

Ver los correos en desarrollo

Se eligió **Mailpit**, un servidor SMTP de pruebas que corre en Docker, no necesita cuenta y no manda nada a internet:

1. `docker compose up -d` levanta Mailpit junto con PostgreSQL (SMTP en el puerto `1025`, interfaz web en el `8025`).
2. En el `.env` se dejan los valores de `.env.example` (`SMTP_HOST=localhost`, `SMTP_PORT=1025`, sin usuario ni contraseña).
3. Los correos aparecen en http://localhost:8025, con su versión HTML y su versión de texto.

Para usar Ethereal, Mailtrap o un SMTP real solo cambian las variables `SMTP_*` y `MAIL_FROM`; el código no se toca.

Correos que envía el API

Correo	Qué lo dispara	A quién le llega	Por qué
Verifica tu correo	`POST /auth/register` (o la vista `/register`) y `POST /auth/resend-verification`	A quien se registra	Confirmar que el correo es suyo antes de dejarle subir audio
Restablece tu contraseña	`POST /auth/forgot-password`, solo si el correo existe	Al dueño de la cuenta	Recuperar el acceso sin ayuda de un administrador

Correo	Qué lo dispara	A quién le llega	Por qué
Tu resumen está listo	`POST /summaries`	Al **dueño de la grabación**, aunque el resumen lo cree un administrador, y solo si su correo está verificado	Avisar que ya puede leer el resumen; lleva el título, la plantilla, un extracto y el enlace a la grabación

Tokens de un solo uso

La verificación del correo y la recuperación de contraseña usan el mismo mecanismo (`src/lib/oneTimeToken.ts`):

Paso	Qué se hace
Generar	`crypto.randomBytes(32).toString("hex")`: 64 caracteres aleatorios. Nunca `Math.random()` ni un JWT con datos del usuario
Guardar	En la tabla `AuthToken` se guarda **el sha256 del token**, su propósito, el usuario y la fecha de vencimiento. El token en claro solo viaja en el correo
Usar	Se busca por el hash y se marca como usado en una sola operación que exige `usedAt` vacío y `expiresAt` en el futuro. Así, dos peticiones al mismo tiempo no pueden usarlo las dos
Invalidar	Pedir un enlace nuevo borra los anteriores sin usar del mismo usuario y propósito

¿Por qué aquí basta sha256 y en las contraseñas no? Una contraseña la elige una persona y se puede adivinar probando millones de opciones, por eso necesita un algoritmo lento como bcrypt. Un token de 32 bytes aleatorios no se puede adivinar, así que basta un hash rápido para que, si alguien lee la base de datos, no pueda usar los enlaces pendientes.

Token	Vigencia	Variable
Verificar correo	24 horas	`VERIFY_TOKEN_TTL`
Restablecer contraseña	1 hora	`RESET_TOKEN_TTL`

Un token inventado, vencido o ya usado responde **400** con un mensaje claro: `Invalid token`, `Token expired` o `Token already used`.

Verificación del correo

- Al registrarse con contraseña, el usuario queda con `emailVerified: false` y recibe el enlace `APP_URL/verify-email?token=…`. `GET /auth/me` incluye `emailVerified`.
- `POST /auth/verify-email` con `{ "token": "…" }` (o abrir el enlace en la vista) marca el correo como verificado y responde 200.
- `POST /auth/resend-verification` (con sesión) invalida el enlace anterior y manda uno nuevo. **Si el correo ya está verificado responde 200 sin enviar nada.**
- Quien entra con Google queda verificado, porque Google exige `email_verified`.

- **Qué puede hacer un usuario sin verificar:** iniciar sesión, ver y editar su perfil, y ver, editar y borrar sus grabaciones. **No puede subir ni procesar audio** (`POST /recordings` y `POST /recordings/:id/process`). Esas rutas usan el middleware reutilizable `requireVerifiedEmail`, que responde 403 con un código que el cliente puede distinguir:

```
{ "error": { "message": "Verifica tu correo para realizar esta acción", "code":
"EMAIL_NOT_VERIFIED", "requestId": "…" } }
```

Recuperar la contraseña

- `POST /auth/forgot-password` con `{ "email": "…" }` responde **siempre 200 con el mismo mensaje** («Si el correo está registrado, recibirás un enlace»), exista o no el correo, y no espera a que salga el correo. Así ni la respuesta ni su tiempo revelan qué correos tienen cuenta.
- Si el correo existe, envía el enlace `APP_URL/reset-password?token=…`, que vence en 1 hora. Pedir otro enlace invalida el anterior.
- `POST /auth/reset-password` con `{ "token": "…", "password": "…" }` valida la contraseña con las mismas reglas que el registro, la guarda con bcrypt, marca el token como usado y responde 200. Después, la contraseña anterior ya no sirve y la nueva sí.
- Un usuario que solo entra con Google (sin contraseña) puede usar este flujo para **crear** una. Restablecer la contraseña también deja el correo verificado, porque abrir el enlace prueba que el correo es suyo.

Logging

Qué se registra

- **Un solo logger:** `src/lib/logger.ts` configura pino y se reutiliza en todo el proyecto. En `src/` no queda ningún `console.log`.
- **Una línea por petición:** el middleware `requestLogger` (pino-http) escribe método, ruta, código de respuesta y duración en milisegundos, por ejemplo `POST /api/v1/auth/login 401 167ms`.
- **Id de petición:** se toma del encabezado `X-Request-Id` si el cliente lo manda (y es corto y seguro) o se genera con `crypto.randomUUID()`. Aparece en todas las líneas de esa petición, se regresa en el encabezado `X-Request-Id` y va en el JSON de error como `requestId`.
- **Errores:** el `errorHandler` escribe cada error 500 con su *stack* y el id de petición. La respuesta nunca incluye el *stack*.
- **Formato:** en producción (`NODE_ENV=production`) cada evento sale como una línea JSON; en desarrollo, en un formato legible de una línea.

Nivel	Cuándo se usa
`error`	Fallas: errores 500 con su *stack*
`warn`	Lo sospechoso o recuperable: respuestas 4xx, login fallido, correo que no salió
`info`	Eventos importantes: arranque, registro, correo enviado, correo verificado, contraseña restablecida, respuestas 2xx
`debug`	Detalle de desarrollo: por qué se rechazó una petición o qué error de Prisma se tradujo

El nivel mínimo se elige con `LOG_LEVEL` en el `.env` (`info` por defecto). Para ver más detalle: `LOG_LEVEL=debug` .

Qué no se registra

Nunca se escriben contraseñas, tokens de sesión, tokens de verificación o de restablecimiento, el encabezado `Authorization` ni las cookies. No se confía en que nadie los imprima: el logger tiene configurada la **redacción** (`redact`) de `password` , `passwordHash` , `token` , `accessToken` , `authorization` , `cookie` y `set-cookie` , que se reemplazan por `[Redacted]` . Además, los parámetros `token` , `code` y `state` se ocultan en las URLs que se escriben en el log (`/verify-email?token=[Redacted]`). El log del correo lleva el tipo de correo y el id del usuario, nunca el enlace.

Documentación del API (Swagger)

- `GET /docs` muestra la documentación interactiva (Swagger UI) y `GET /docs/openapi.json` la especificación OpenAPI 3. Ambas son públicas.
- Cada endpoint se documenta con un comentario `@openapi` junto a su ruta, en el archivo de rutas de su módulo. La configuración general (`info` , `servers` , `tags` , `securitySchemes`) está en `src/docs/swagger.ts` , y los esquemas, parámetros y respuestas compartidos en `src/docs/schemas.ts` , reutilizados con `$ref` . Los esquemas de usuario no incluyen la contraseña.
- Los endpoints de las entregas 1, 2 y 3 están documentados y agrupados por módulo, con sus parámetros, ejemplos y cada respuesta posible.
- La autenticación se declara como `bearerAuth` y se aplica solo a las rutas protegidas. Con **Authorize** se pega el `accessToken` de `POST /auth/login` y se pueden probar las rutas protegidas con *Try it out*.
- La especificación se genera tanto con `npm run dev` (archivos `.ts`) como con `npm run build && npm start` (archivos `.js`), porque el `tsconfig.json` conserva los comentarios. En esta entrega se corrigió que en **Windows** `/docs` salía vacío: la ruta de búsqueda usaba `\` y la librería de *glob* solo entiende `/` .

Roles y permisos

Rol	Descripción
Invitado	Persona sin sesión. Solo puede registrarse, iniciar sesión y consultar `/health`.
Usuario (USER)	Rol por defecto al registrarse. Administra únicamente sus propias grabaciones, transcripciones y resúmenes.
Administrador (ADMIN)	Soporte y moderación: ve y administra los recursos de todos los usuarios y puede eliminar usuarios.

El rol solo lo cambia un administrador con `PATCH /users/:id` y `{ "role": "ADMIN" }`. El primer administrador viene del seed (Sebastián Macías) o se asigna directamente en la base de datos.

Matriz de permisos

«Propios» significa que la acción solo se permite sobre recursos del mismo usuario; sobre los de otro responde 403.

Acción	Invitado	Usuario	Admin	Dónde se aplica
Registrarse e iniciar sesión	Sí	Sí	Sí	Rutas públicas
Ver su perfil (`GET /auth/me`)	No	Sí	Sí	`requireAuth`
Listar usuarios	No	No	Sí	`requireRole("ADMIN")`
Ver o editar usuario	No	Propio	Todos	`requireSelfOrAdmin`
Cambiar el rol de un usuario	No	No	Sí	`updateUser`
Eliminar usuario	No	No	Sí	`requireRole("ADMIN")`
Listar grabaciones	No	Propias	Todas	Filtro en `getRecordings`
Crear grabación	No	A su nombre	Para cualquier usuario	`assertCanAccess`
Subir o procesar audio	No	Con el correo verificado	Con el correo verificado	`requireVerifiedEmail` (403 `EMAIL_NOT_VERIFIED`)
Ver, editar, procesar o eliminar grabación	No	Propias	Todas	`findRecordingOr404`
Transcripciones y resúmenes	No	De sus grabaciones	Todos	`assertCanAccess`
Vistas web	No	Sus datos	Todos los datos	`viewAuth` y los mismos filtros

Orden de las respuestas: primero 401 (sin sesión válida) y 400 (datos inválidos). En grabaciones, transcripciones y resúmenes sigue 404 (no existe) y luego 403 (no es tuyo). En `/users/:id`, un

usuario normal recibe 403 con cualquier id que no sea el suyo, exista o no.

REST API

Todas las rutas usan el prefijo `/api/v1` (excepto `/health` y las vistas). Las respuestas exitosas tienen la forma `{ data, message }` y los errores `{ error: { message, details?, code?, requestId } }`. `requestId` es el mismo del encabezado `X-Request-Id` y de las líneas del log de esa petición.

Endpoints

Módulo	Método y ruta	Acceso	Respuestas
Health	`GET /health`	Público	200
Docs	`GET /docs` · `GET /docs/openapi.json`	Público	Swagger UI y la especificación OpenAPI 3
Auth	`POST /auth/register`	Público	201 (rol USER); 400; 409 correo duplicado
Auth	`POST /auth/login`	Público	200 con `accessToken`; 400; 401 genérico
Auth	`GET /auth/me`	Sesión	200 (incluye `emailVerified`); 401
Auth	`POST /auth/verify-email`	Público	200; 400 token inválido, vencido o usado
Auth	`POST /auth/resend-verification`	Sesión	200 (si ya está verificado, 200 sin enviar nada); 401
Auth	`POST /auth/forgot-password`	Público	Siempre 200 con el mismo mensaje
Auth	`POST /auth/reset-password`	Público	200; 400 token inválido, vencido o usado, o contraseña que no cumple las reglas
Users	`GET /users`	ADMIN	200; 401; 403
Users	`GET /users/:id` · `PATCH /users/:id`	Propio o ADMIN	200; 400; 401; 403; 404; 409 correo duplicado
Users	`DELETE /users/:id`	ADMIN	204; 401; 403; 404; 409 con grabaciones
Recordings	`GET /recordings` · `GET /recordings/:id`	Propias o ADMIN	200; 400; 401; 403; 404
Recordings	`GET /recordings/:id/summaries` · `/transcription`	Propias o ADMIN	200 (rutas anidadas); 401; 403; 404

Módulo	Método y ruta	Acceso	Respuestas
Recordings	`POST /recordings`	Propias o ADMIN, con correo verificado	201; 400; 401; 403 (incluye `EMAIL_NOT_VERIFIED`); 404
Recordings	`PATCH /:id` · `DELETE /:id`	Propias o ADMIN	200 / 204; 400; 401; 403; 404
Recordings	`POST /recordings/:id/process`	Propias o ADMIN, con correo verificado	202; 401; 403 (incluye `EMAIL_NOT_VERIFIED`); 404
Transcriptions	`GET` · `GET /:id` · `POST` · `PATCH /:id` · `DELETE /:id`	De sus grabaciones o ADMIN	200 / 201 / 204; 400; 401; 403; 404; 409 si la grabación ya tiene transcripción
Summaries	`GET` · `GET /:id` · `POST` · `PATCH /:id` · `DELETE /:id`	De sus grabaciones o ADMIN	200 / 201 / 204; 400; 401; 403; 404

Los listados aceptan filtros validados: `?status=&userId=` en recordings, `?recordingId=&template=` en summaries y `?recordingId=` en transcriptions.

Validaciones y errores

- El middleware reutilizable `validate({ body, params, query })` corre antes del controlador con esquemas zod: obligatorios, tipos, longitudes, formatos (correo, UUID, código de idioma), rangos y enums. Un campo desconocido también es error.
- Una petición inválida responde 400 e indica en `details` qué campo falló y por qué.
- El `errorHandler` central traduce los errores de Prisma: P2002 → 409, P2025 → 404, P2003 → 409, P2023 → 400. Nunca responde con el stack ni con el código de la BD; ese detalle se escribe en el log con el id de la petición.
- Las rutas inexistentes responden 404 en JSON.

Vistas

Ruta	Vista	Acceso
`GET /register` · `POST /register`	Crear cuenta: nombre, correo y contraseña. Valida cada campo, responde 409 si el correo ya existe, crea un `USER`, abre la sesión y redirige al inicio	Público
`GET /login` · `POST /login` · `POST /logout`	Iniciar y cerrar sesión, con enlace al registro y botón de Google si está configurado	Público
`GET /auth/google` · `GET /google_confirm`	Inicio y regreso del login con Google	Público

Ruta	Vista	Acceso
`GET /verify-email?token=…`	Destino del enlace del correo de verificación: verifica la cuenta y avisa si el enlace no sirve	Público
`GET /forgot-password` · `POST /forgot-password`	Pedir el enlace para restablecer la contraseña (enlace «¿Olvidaste tu contraseña?» del login)	Público
`GET /reset-password?token=…` · `POST /reset-password`	Formulario para escribir la contraseña nueva	Público
`POST /resend-verification`	Botón «Reenviar enlace» del aviso de correo sin verificar	Sesión
`GET /`	Inicio: conteos y últimas grabaciones	Sesión (sus datos)
`GET /recordings` · `GET /recordings/:id`	Listado con filtro por estado y detalle con transcripción y resúmenes	Sesión (propias, o todas si es ADMIN)
`GET /users`	Usuarios con su rol y su número de grabaciones	ADMIN

Sin sesión, las vistas protegidas redirigen a `/login`. La barra muestra el nombre y el rol, y oculta «Usuarios» a quien no es administrador.

Datos de prueba y colección

Seed

`npm run seed` (`prisma/seed.ts`) limpia las 5 tablas y vuelve a cargar los mismos registros en una transacción, por lo que se puede correr varias veces sin errores ni duplicados: 7 usuarios, 8 grabaciones (con los 4 estados), 5 transcripciones y 6 resúmenes de las 3 plantillas. Todos los usuarios usan la contraseña `Recall2026!`, guardada con bcrypt, y tienen el correo verificado para que las pruebas no se bloqueen, excepto Laura Sánchez (`laura.sanchez@recall.app`), que queda con el correo **sin verificar** para probar el 403 `EMAIL_NOT_VERIFIED`. Sebastián Macías es `ADMIN`; los demás, `USER`.

Colección de Postman

`docs/api.postman_collection.json` tiene **129 peticiones** en 11 carpetas: Health, Auth, Verificación de correo, Recuperar contraseña, Users, Recordings, Transcriptions, Summaries, Correo del proyecto (resumen listo), Permisos y Errores generales. La URL base está en la variable `{{baseUrl}}`, y los tokens de admin y de usuario se guardan solos al correr la carpeta Auth.

Las tres carpetas de correo (34 peticiones) **necesitan Mailpit** (`docker compose up -d`): el token solo viaja en el correo, así que la colección lo lee de la API de Mailpit (variable `{{mailpitUrl}}`). Registran un usuario nuevo, leen su correo, usan el enlace y comprueban los casos de error: token

inexistente, token ya usado, enlace anterior invalidado por un reenvío, `forgot-password` con un correo inexistente (el mismo 200 genérico), contraseña débil, la contraseña anterior que deja de servir y la acción sin verificar (403 `EMAIL_NOT_VERIFIED`, con la usuaria del seed). El usuario que crean se borra al final. El token expirado no se automatiza: se prueba a mano con `VERIFY_TOKEN_TTL=5s`.

Caso de seguridad	Respuesta esperada
Registro con `role: "ADMIN"` en el body	400, indica el campo `role`
Login sin contraseña	400, indica el campo `password`
Contraseña incorrecta y correo inexistente	401 con exactamente la misma respuesta
La contraseña nunca aparece en las respuestas	Registro, login, perfil y usuarios no contienen `password`
Sin token (perfil, crear grabación, eliminar usuario y listados)	401 `Unauthorized`
Token alterado: `sub` cambiado al del admin	401 `Invalid token`
Token alterado: firma modificada	401 `Invalid token`
Token sin firma (`alg: none`)	401 `Invalid token`
Token firmado con otra clave	401 `Invalid token`
Usuario intenta acciones de otro o de admin	403

Los tokens alterados se generan en el *pre-request* a partir de un token real, así que la prueba siempre ataca un token válido del mismo servidor. El **token expirado** no se puede automatizar en Postman sin esperar a que venza, así que se prueba a mano con `JWT_EXPIRES_IN` corto (ver Evidencia 4).

Pruebas y evidencia

Las pruebas de las evidencias 1 a 9 (salvo la 2, repetida el 11 de noviembre con la colección final) se corrieron el 5 de noviembre de 2026 en una base PostgreSQL 17 desechable, con las 5 migraciones aplicadas y los datos del seed. Las imágenes de terminal son la salida real de los comandos, y las de las vistas son capturas del navegador con la aplicación en ejecución.

Entrega 2: autenticación y permisos

Calidad y migraciones

Lint, typecheck, migraciones, seed y build

```
$ npm run lint
OK sin errores

$ npm run typecheck
OK sin errores

$ npx prisma migrate deploy
5 migrations found in prisma/migrations
No pending migrations to apply.

$ npx prisma migrate diff --from-config-datasource --to-schema prisma/schema.prisma
No difference detected.

$ npm run seed
Seed listo: 6 users, 8 recordings, 5 transcriptions, 6 summaries

$ npm run build
OK compilado en dist/
```

Evidencia 1 · ESLint y TypeScript sin errores; la base queda idéntica a `schema.prisma`

Colección de Postman

Corrieron las 129 peticiones con newman: 242 aserciones y ninguna falla (newman cuenta 133 requests porque 4 son consultas a Mailpit de los scripts). Esta evidencia se repitió el 11 de noviembre de 2026, con el código final de `main` y Mailpit en Docker. La imagen muestra una selección de los casos de seguridad, las tres carpetas de correo y el resumen de toda la colección.

Colección de Postman: casos de seguridad, flujos de correo y resumen de las 129 peticiones

```
$ npx newman run docs/api.postman_collection.json

# Casos de seguridad (selección)
OK   Registrar — intenta asignarse role ADMIN (400) -> 400 (3 pruebas)
OK   Login — contraseña incorrecta (401) -> 401 (2 pruebas)
OK   Login — usuario inexistente (401) -> 401 (3 pruebas)
OK   Token alterado: cambia el sub al del admin (401) -> 401 (2 pruebas)
OK   Token sin firma, alg none (401) -> 401 (2 pruebas)
OK   Usuario — listar usuarios (403) -> 403 (1 pruebas)
OK   Usuario — ver grabación ajena (403) -> 403 (1 pruebas)

# Verificación de correo
OK   Registrar usuario (queda sin verificar) -> 201 (3 pruebas)
OK   Login sin verificar (no se bloquea) -> 200 (2 pruebas)
OK   Mi perfil incluye emailVerified: false -> 200 (2 pruebas)
OK   Mailpit: correo de verificación llegó -> 200 (5 pruebas)
OK   Reenviar verificación — sin sesión (401) -> 401 (1 pruebas)
OK   Reenviar verificación (invalida el enlace anterior) -> 200 (2 pruebas)
OK   Mailpit: llegó un enlace nuevo -> 200 (2 pruebas)
OK   Verificar correo — el enlace anterior ya no sirve (400) -> 400 (3 pruebas)
OK   Verificar correo — token inexistente (400) -> 400 (3 pruebas)
OK   Verificar correo — token con formato inválido (400) -> 400 (2 pruebas)
OK   Verificar correo — sin token (400) -> 400 (1 pruebas)
OK   Verificar correo -> 200 (2 pruebas)
OK   Verificar correo — token ya usado (400) -> 400 (3 pruebas)
OK   Mi perfil: emailVerified pasó a true -> 200 (2 pruebas)
OK   Reenviar verificación — ya verificado (200, no envía nada) -> 200 (2 pruebas)
OK   Login usuaria sin verificar del seed (Laura) -> 200 (2 pruebas)
OK   Sin verificar — crear grabación (403 EMAIL_NOT_VERIFIED) -> 403 (4 pruebas)
OK   Sin verificar — procesar grabación (403 EMAIL_NOT_VERIFIED) -> 403 (4 pruebas)
OK   Sin verificar — sí puede ver sus grabaciones -> 200 (1 pruebas)

# Recuperar contraseña
OK   Olvidé mi contraseña — correo inexistente (200 genérico) -> 200 (2 pruebas)
OK   Olvidé mi contraseña — correo registrado (200 genérico) -> 200 (2 pruebas)
OK   Olvidé mi contraseña — correo inválido (400) -> 400 (1 pruebas)
OK   Mailpit: correo de restablecimiento llegó -> 200 (5 pruebas)
OK   Restablecer — contraseña débil (400, el token no se gasta) -> 400 (2 pruebas)
OK   Restablecer — token inexistente (400) -> 400 (3 pruebas)
OK   Restablecer — sin token (400) -> 400 (1 pruebas)
OK   Restablecer contraseña -> 200 (2 pruebas)
OK   Restablecer — token ya usado (400) -> 400 (3 pruebas)
OK   Login con la contraseña anterior (401) -> 401 (3 pruebas)
OK   Login con la contraseña nueva -> 200 (1 pruebas)
OK   Limpieza: eliminar el usuario del flujo -> 204 (1 pruebas)

# Correo del proyecto (resumen listo)
OK   Crear resumen en una grabación de Ana (como admin) -> 201 (1 pruebas)
OK   Mailpit: el correo "Tu resumen está listo" llegó al dueño -> 200 (5 pruebas)
OK   Limpieza: eliminar el resumen creado -> 204 (1 pruebas)

Resumen de toda la colección
  peticiones (items)   ejecutadas:  129   fallidas: 0
  requests             ejecutadas:  133   fallidas: 0   (4 son consultas a Mailpit de los scripts)
  test-scripts         ejecutadas:  129   fallidas: 0
  prerequest-scripts   ejecutadas:   10   fallidas: 0
  assertions           ejecutadas:  242   fallidas: 0
```

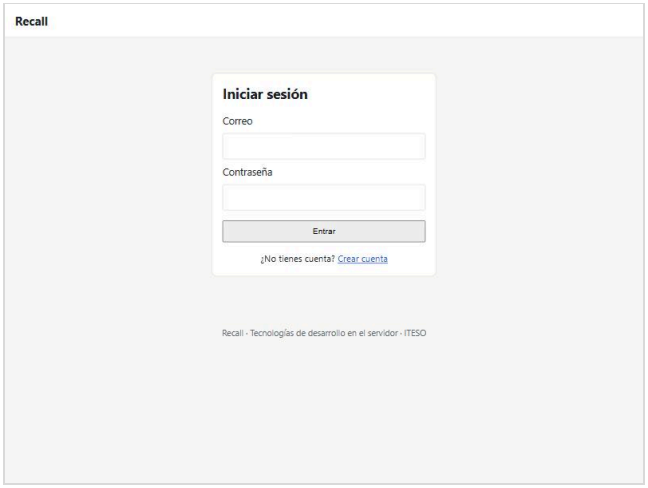
Evidencia 2 · Casos de seguridad, flujos de correo y resumen de las 129 peticiones

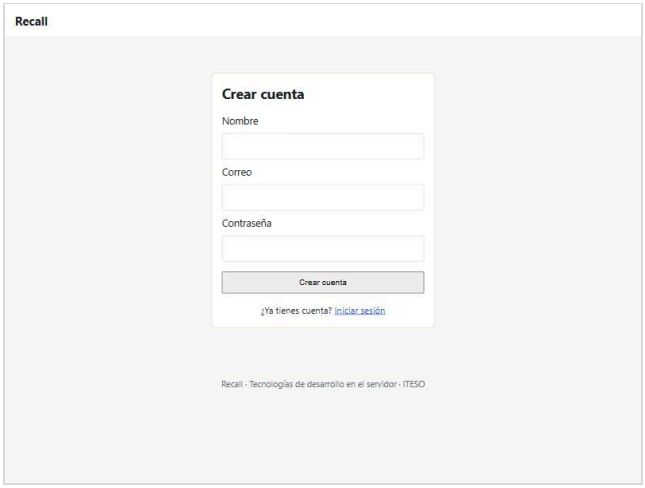
Vista de registro

Vista de registro probada con curl

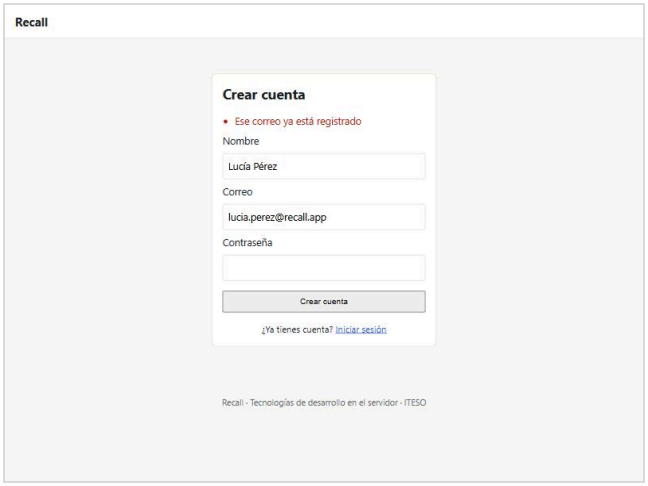
```
Vista de registro (POST /register)
GET /register                                   -> 200
Campos válidos                                  -> 302 http://localhost:3993/
Con la cookie recibida: GET /recordings         -> 200
Mismo correo otra vez                           -> 409 Ese correo ya está registrado
Correo inválido y contraseña corta              -> 400 Escribe un correo válido La contraseña debe tener entre 8 y 72 caracteres
Formulario alterado con role=ADMIN              -> 400 El formulario tiene campos no permitidos
```

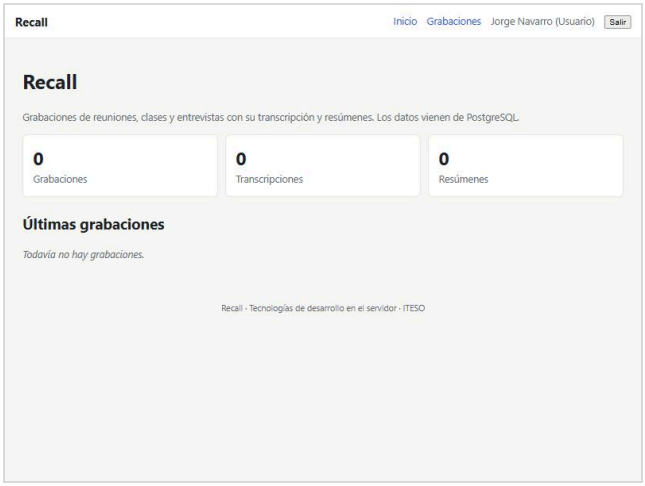
Evidencia 3 · El registro crea la cuenta e inicia sesión; rechaza el correo duplicado, los datos inválidos y el formulario alterado con `role=ADMIN`





Evidencia 3a · Login con el enlace «Crear cuenta» y formulario de registro





Evidencia 3b · Correo ya registrado (409) y cuenta nueva con la sesión abierta como Usuario

Token expirado

Token expirado (JWT_EXPIRES_IN=3s)

```
$ JWT_EXPIRES_IN=3s npm start

$ POST /api/v1/auth/login  (Ana)
token recibido; vigencia: 3 segundos

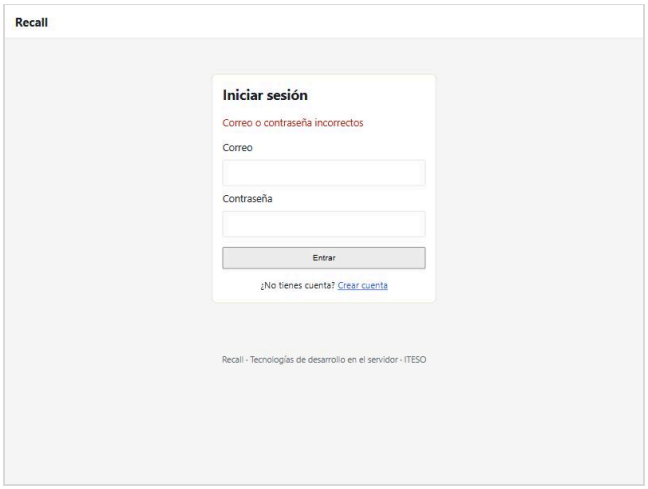
$ GET /api/v1/auth/me  (de inmediato)
[HTTP 200] responde el perfil de Ana

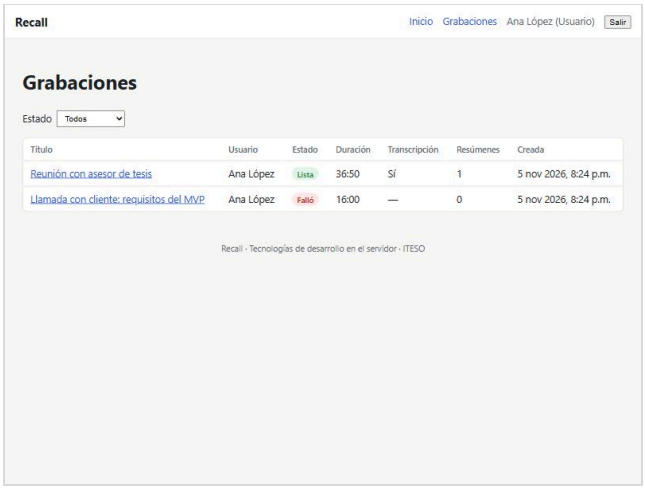
$ sleep 5

$ GET /api/v1/auth/me  (5 s después, mismo token)
{"error":{"message":"Token expired"}} [HTTP 401]
```

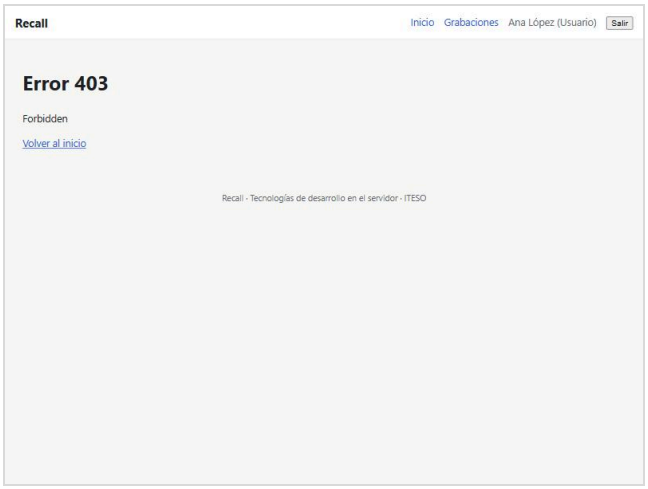
Evidencia 4 · Con `JWT_EXPIRES_IN=3s`, el mismo token funciona al inicio y 5 segundos después responde 401

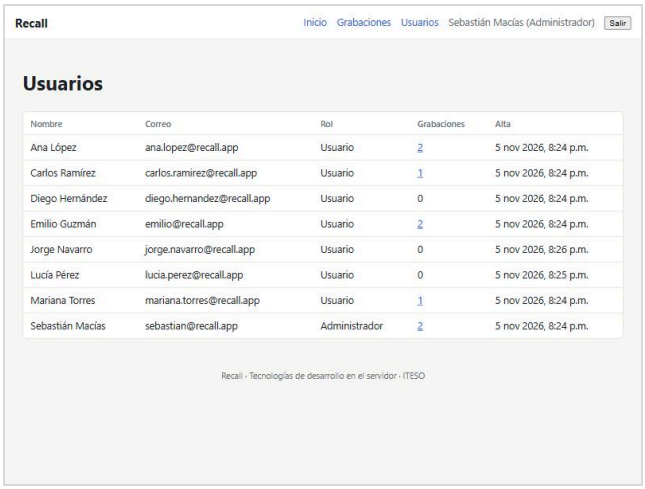
Login y permisos en las vistas



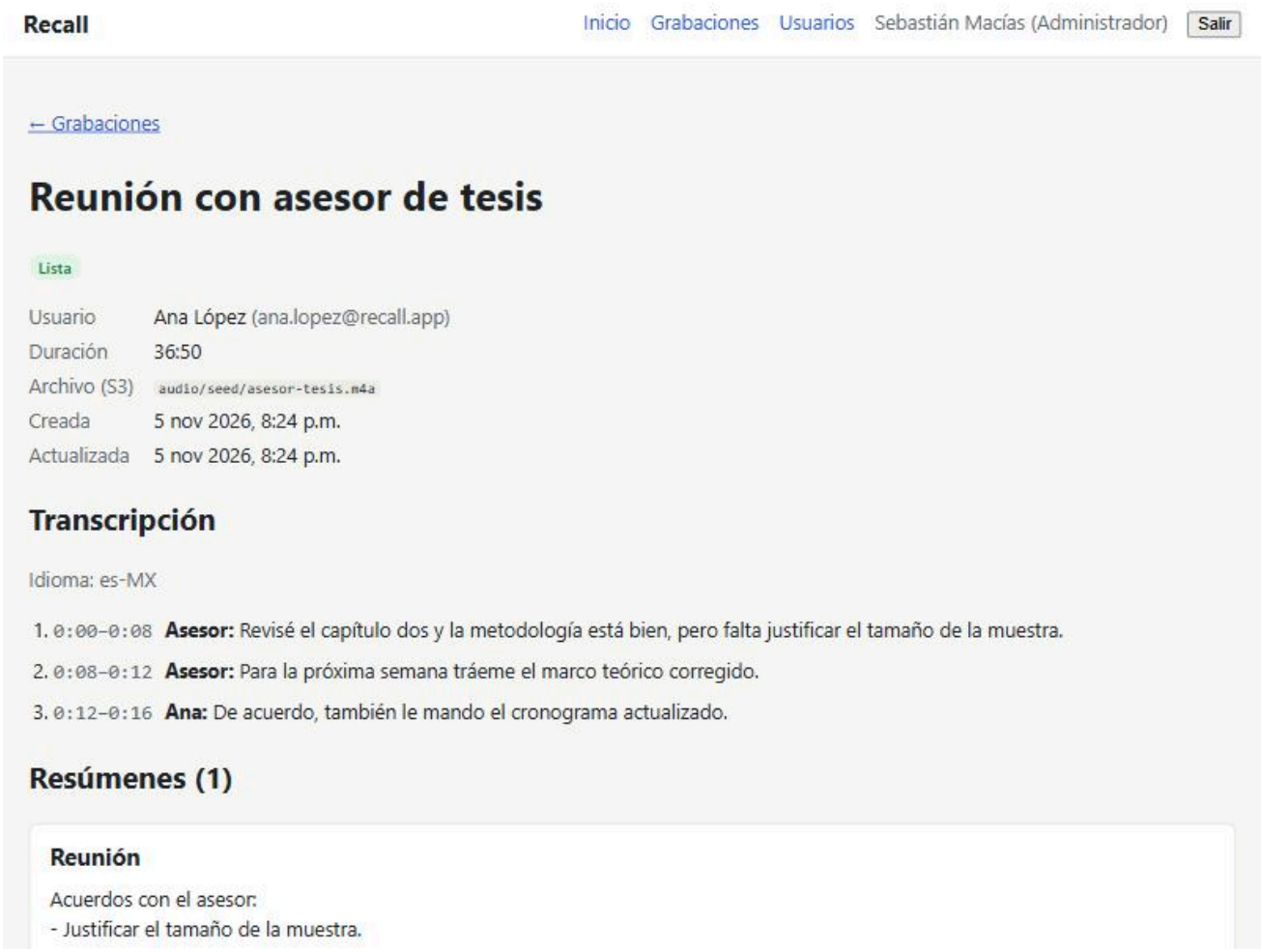


Evidencia 5 · Contraseña incorrecta con mensaje genérico, y Ana López (USER) ve solo sus 2 grabaciones



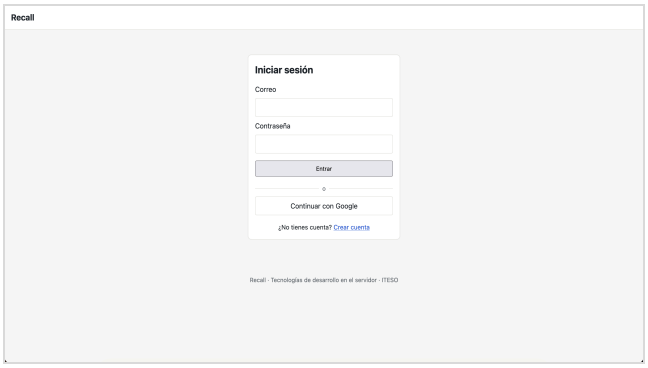


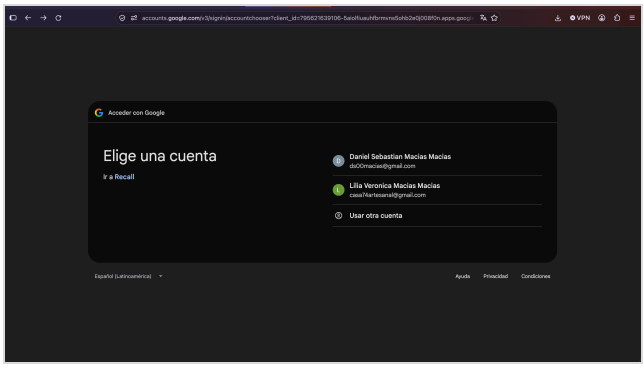
Evidencia 6 · Ana López recibe 403 en «Usuarios»; Sebastián Macías (ADMIN) ve la lista con los roles



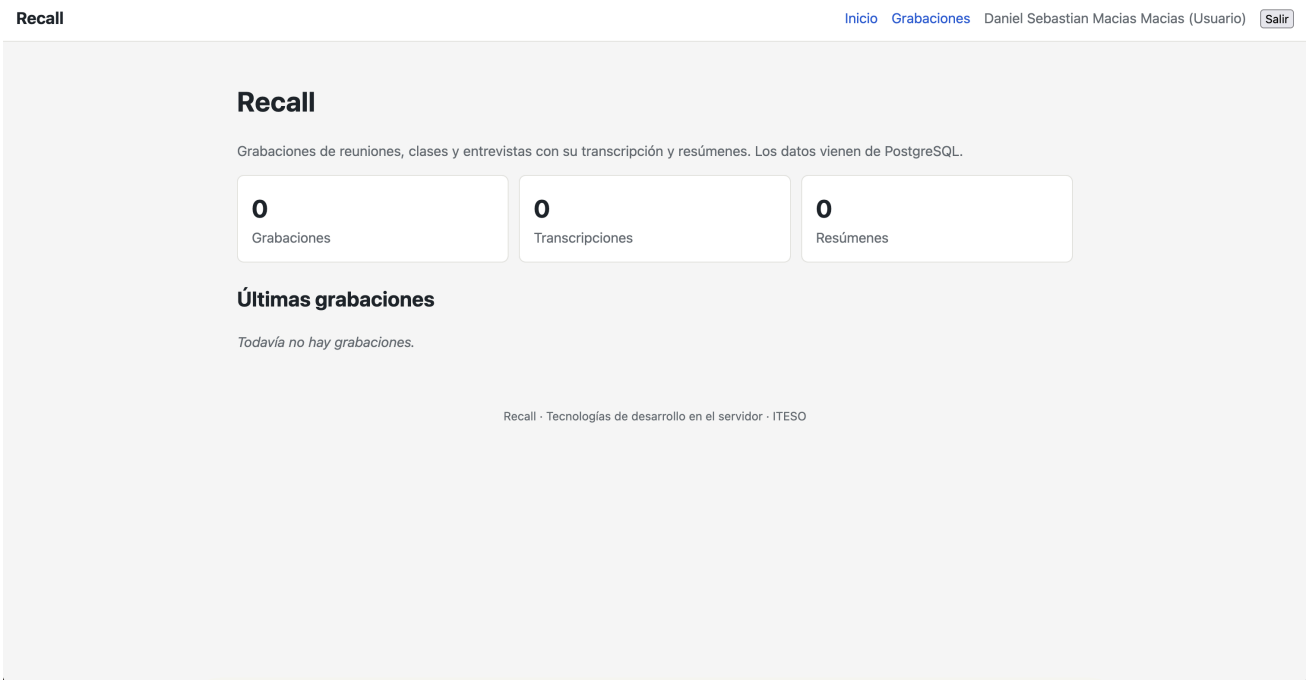
Evidencia 7 · El administrador abre una grabación de Ana López

Login con Google





Evidencia 8 · El botón «Continuar con Google» en `/login` redirige a Google, que pide elegir la cuenta



Evidencia 9 · Al volver del callback la sesión queda abierta con el rol Usuario (`USER`), el rol por defecto: nunca viene de Google

> **Credenciales:** el login con Google requiere las credenciales OAuth del proyecto, que no se suben al repositorio. Se prueba en local con `GOOGLE_CLIENT_ID` , `GOOGLE_CLIENT_SECRET` y `GOOGLE_CALLBACK_URL` en el `.env` (ver README).

Entrega 3: correos, logging y Swagger

Las evidencias 10 a 17 se corrieron el 10 de noviembre de 2026 con `npm run build && npm start` , una base PostgreSQL 17 desechable con las 6 migraciones y el seed, y Mailpit en Docker. Las imágenes de terminal son la salida real de los comandos y las de Mailpit y Swagger son capturas del navegador.

Flujos de correo de punta a punta

Un script recorre los tres flujos contra el API y lee los enlaces reales desde la API de Mailpit: registro y verificación, recuperación de contraseña y el correo «Tu resumen está listo». Resultado: **28 verificaciones, 28 correctas**.

Flujos de correo de punta a punta (28 verificaciones)

```
$ node flujos.mjs   (registro, verificación, recuperación y resumen listo contra Mailpit)

Usuaria de prueba: valeria.ruiz.43501@recall.app

-- Registro y verificación de correo --
OK    POST /auth/register -> 201, emailVerified=false
OK    Llega «Verifica tu correo» con HTML y texto -> Recall: verifica tu correo
OK    El enlace usa APP_URL -> http://localhost:3990/verify-email?token=…
OK    El token tiene 64 caracteres hex (32 bytes)
OK    Iniciar sesión sin verificar está permitido -> 200
OK    GET /auth/me incluye emailVerified -> emailVerified=false
OK    POST /recordings sin verificar -> 403 EMAIL_NOT_VERIFIED -> 403 EMAIL_NOT_VERIFIED
      cuerpo: {"error":{"message":"Verifica tu correo para realizar esta acción","code":"EMAIL_NOT_VERIFIED","requestId":"35b1abda-5a89-4b60-
8d52-abaa21c4c220"}}
OK    La respuesta trae X-Request-Id -> 35b1abda-5a89-4b60-8d52-abaa21c4c220
OK    POST /auth/resend-verification -> 200 Verification email sent
OK    Llega un segundo enlace distinto
OK    El enlace anterior quedó invalidado -> 400 Invalid token
OK    Token inventado -> 400 Invalid token
OK    Token válido -> correo verificado -> 200 Email verified
OK    El mismo token otra vez -> 400 Token already used
OK    Reenviar ya verificado: 200 sin mandar correo -> 200 Email already verified, correos: 2
OK    POST /recordings ya verificada -> 201

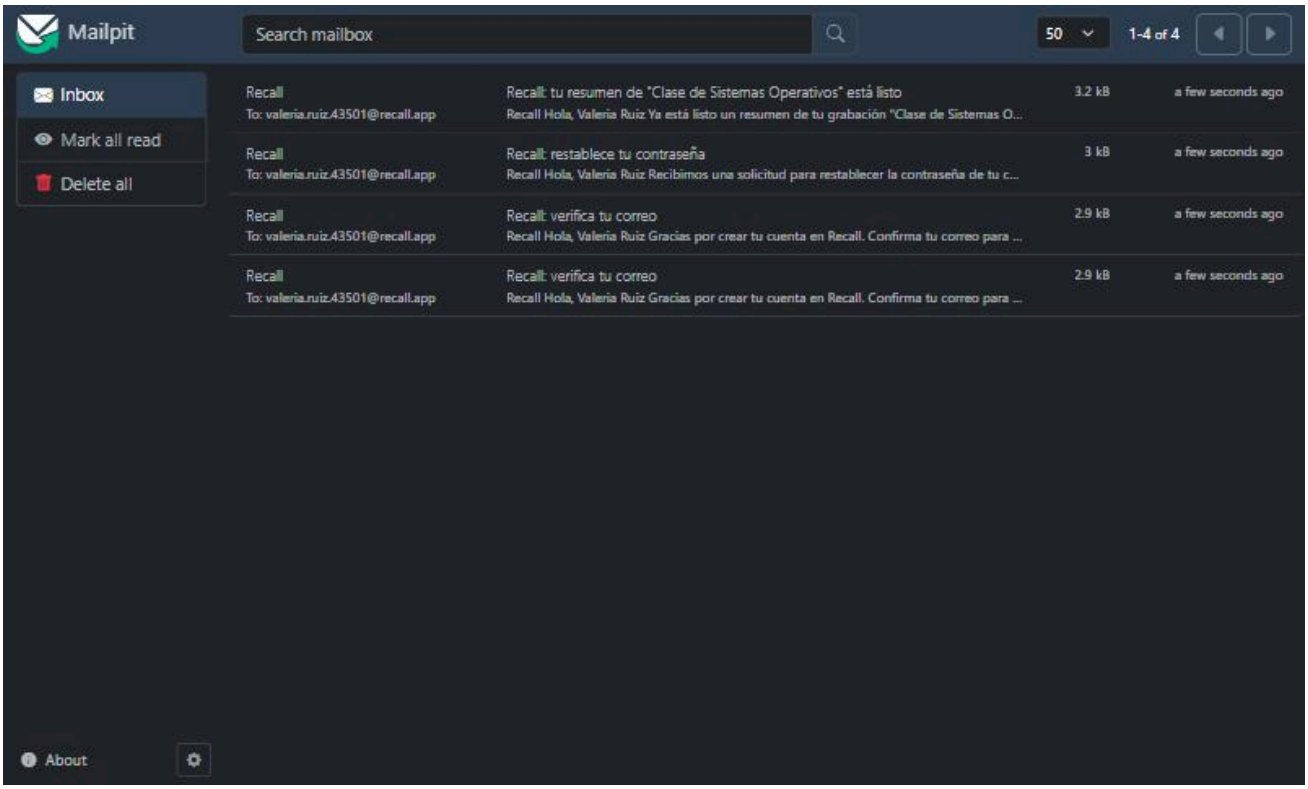
-- Recuperar contraseña --
OK    forgot-password con correo existente -> 200 "Si el correo está registrado, recibirás un enlace"
OK    forgot-password con correo inexistente: misma respuesta -> 200 "Si el correo está registrado, recibirás un enlace"
OK    Llega «Restablece tu contraseña» -> Recall: restablece tu contraseña
OK    No se mandó nada al correo inexistente
OK    Contraseña que no cumple las reglas -> 400 Must be at least 8 characters
OK    reset-password con token válido -> 200 Password updated
OK    El mismo token otra vez -> 400 Token already used
OK    Login con la contraseña anterior -> 401
OK    Login con la contraseña nueva -> 200

-- Correo propio: «Tu resumen está listo» --
OK    Un admin crea un resumen en la grabación de Valeria -> 201
OK    Le llega a la dueña de la grabación, no al admin -> Recall: tu resumen de "Clase de Sistemas Operativos" está listo
OK    No le llegó copia al admin

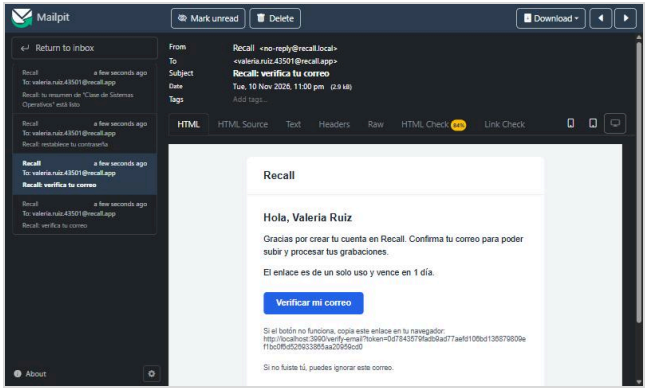
Correos de valeria.ruiz.43501@recall.app en Mailpit: 4
pasaron 28, fallaron 0
```

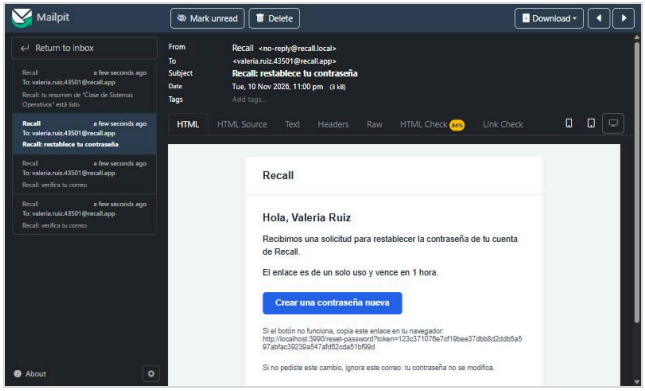
Evidencia 10 · Verificación (token válido, inventado, usado e invalidado al reenviar), recuperación de contraseña y correo propio

Correos en Mailpit

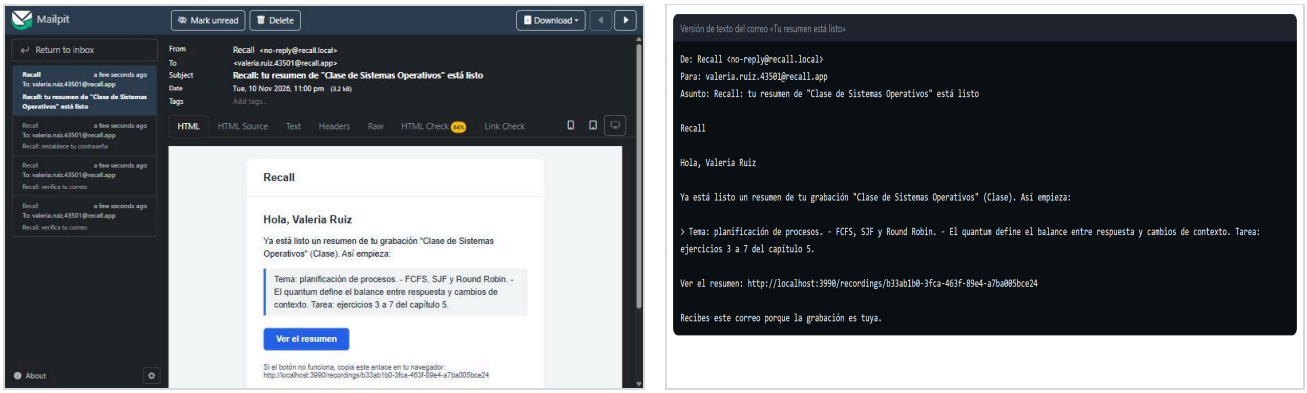


Evidencia 11 · Los correos que recibió la usuaria de prueba: dos de verificación (el segundo, del reenvío), el de recuperación y el de resumen listo



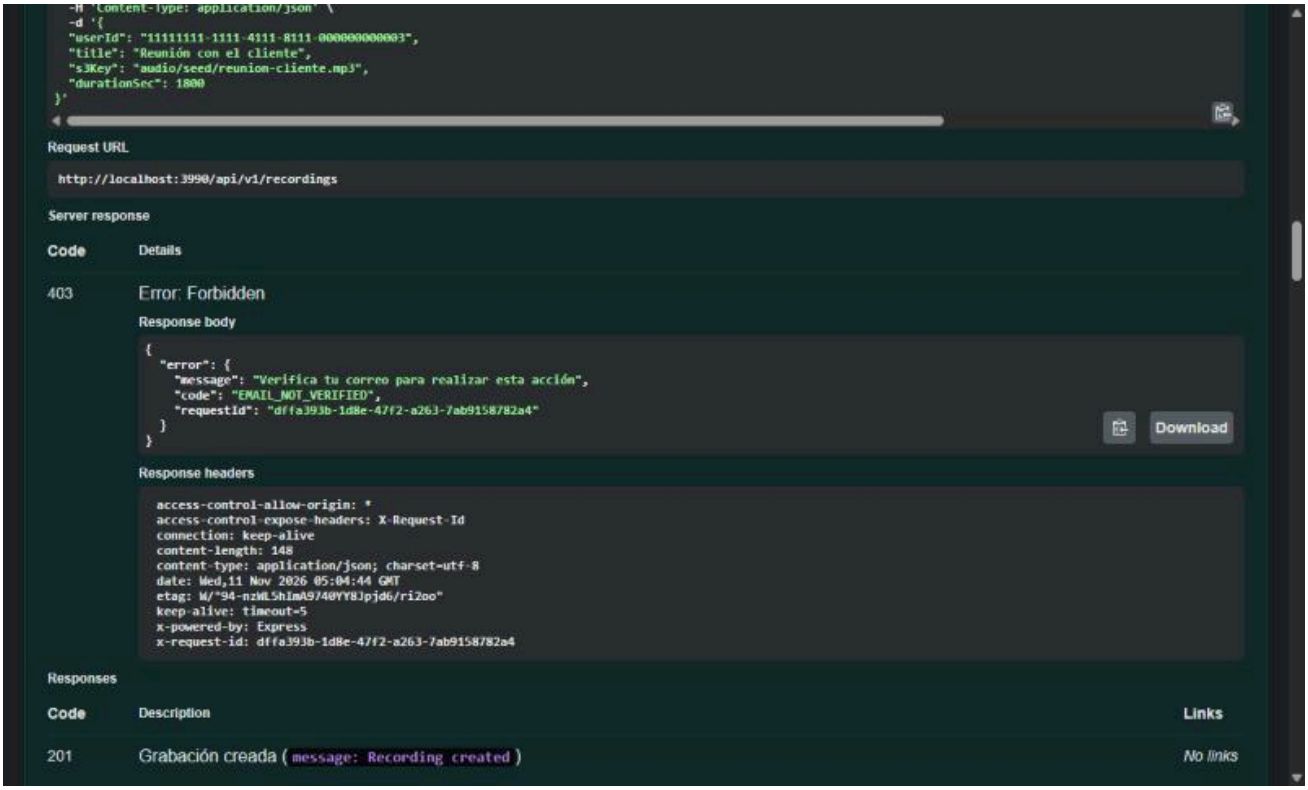


Evidencia 12 · «Verifica tu correo» (vence en 1 día) y «Restablece tu contraseña» (vence en 1 hora), con el enlace armado con `APP_URL`



Evidencia 13 · «Tu resumen está listo» le llega a la dueña de la grabación aunque el resumen lo creó un administrador; a la derecha, su versión de texto

Correo verificado obligatorio, probado desde Swagger



Evidencia 14 · Con **Authorize** y el token de una cuenta sin verificar, *Try it out* en `POST /recordings` responde 403 `EMAIL_NOT_VERIFIED` con el `requestId` igual al encabezado `X-Request-Id`

Logs

Logs en desarrollo: una línea por petición con su id

```
$ npm run dev   (LOG_LEVEL=info, formato legible en desarrollo)

[23:00:23] INFO: User registered {"reqId":"cf4892d6-8892-4ac8-9065-b6d3b6e86af1","userId":"3920ad9f-119f-4c69-9dc9-d6181658c2af"}
[23:00:23] WARN: Email not sent {"reqId":"cf4892d6-8892-4ac8-9065-b6d3b6e86af1","mail":"verify-email","userId":"3920ad9f-119f-4c69-9dc9-
d6181658c2af","reason":"ECONNECTION"}
[23:00:23] INFO: POST /api/v1/auth/register 201 187ms {"reqId":"cf4892d6-8892-4ac8-9065-b6d3b6e86af1"}
[23:00:43] INFO: User registered {"reqId":"24030efd-919a-41eb-9a64-ebccf72cc4fa","userId":"311d3bf0-dfea-4f74-a863-9281b8092be1"}
[23:00:43] INFO: Email sent {"reqId":"24030efd-919a-41eb-9a64-ebccf72cc4fa","mail":"verify-email","userId":"311d3bf0-dfea-4f74-a863-
9281b8092be1"}
[23:00:43] INFO: POST /api/v1/auth/register 201 288ms {"reqId":"24030efd-919a-41eb-9a64-ebccf72cc4fa"}
[23:00:44] INFO: POST /api/v1/auth/login 200 99ms {"reqId":"d04e923b-efe9-4797-afa7-63543adab91a"}
[23:00:44] INFO: GET /api/v1/auth/me 200 6ms {"reqId":"1c9a442c-3ee9-4da5-a78f-5bf56195fe16"}
[23:00:44] WARN: POST /api/v1/recordings 403 4ms {"reqId":"35b1abda-5a89-4b60-8d52-abaa21c4c220"}
[23:00:44] INFO: Email sent {"reqId":"ab1a2c21-287f-4bb3-badb-3fc61769387a","mail":"verify-email","userId":"311d3bf0-dfea-4f74-a863-
9281b8092be1"}
[23:00:44] INFO: POST /api/v1/auth/resend-verification 200 77ms {"reqId":"ab1a2c21-287f-4bb3-badb-3fc61769387a"}
[23:00:44] WARN: POST /api/v1/auth/verify-email 400 10ms {"reqId":"a4dd6613-f08c-44d3-a4d2-ff582e43a971"}
[23:00:44] WARN: POST /api/v1/auth/verify-email 400 7ms {"reqId":"b71f7391-e9e1-4f92-890d-0ff5829d1d5d"}
[23:00:44] INFO: Email verified {"reqId":"214cb5c0-dd26-438b-a846-97cd2b36b480","userId":"311d3bf0-dfea-4f74-a863-9281b8092be1"}
[23:00:44] INFO: POST /api/v1/auth/verify-email 200 11ms {"reqId":"214cb5c0-dd26-438b-a846-97cd2b36b480"}
[23:00:44] WARN: POST /api/v1/auth/verify-email 400 8ms {"reqId":"b35cd391-fe86-4d81-af09-a5557c9afd37"}
[23:00:44] INFO: POST /api/v1/auth/resend-verification 200 6ms {"reqId":"1b48df17-b450-4b45-9a32-10b2cac97d9c"}
[23:00:44] INFO: POST /api/v1/recordings 201 11ms {"reqId":"96d2fc14-2323-47af-aa53-f8cef3a516b8"}
[23:00:44] INFO: POST /api/v1/auth/forgot-password 200 0ms {"reqId":"9e274446-0ac0-4d52-91c2-1d21e44a0a58"}
[23:00:44] INFO: POST /api/v1/auth/forgot-password 200 1ms {"reqId":"4a2e3808-5b91-4953-a16e-148a73bb6af7"}
[23:00:44] INFO: Email sent {"reqId":"9e274446-0ac0-4d52-91c2-1d21e44a0a58","mail":"reset-password","userId":"311d3bf0-dfea-4f74-a863-
9281b8092be1"}
[23:00:49] WARN: POST /api/v1/auth/reset-password 400 1ms {"reqId":"0d2ea2aa-668f-44ae-9e76-8e73b534c9c4"}
[23:00:49] INFO: Password reset {"reqId":"3303ad46-0c57-492a-a1bd-bb0bc7b32752","userId":"311d3bf0-dfea-4f74-a863-9281b8092be1"}
[23:00:49] INFO: POST /api/v1/auth/reset-password 200 173ms {"reqId":"3303ad46-0c57-492a-a1bd-bb0bc7b32752"}
[23:00:50] WARN: POST /api/v1/auth/reset-password 400 149ms {"reqId":"34216984-d431-40e2-ab56-12e9a39e90b7"}
[23:00:50] WARN: Login failed {"reqId":"904a29f1-9037-48d5-ae8a-f9e46c6192e1","email":"valeria.ruiz.43501@recall.app"}
[23:00:50] WARN: POST /api/v1/auth/login 401 167ms {"reqId":"904a29f1-9037-48d5-ae8a-f9e46c6192e1"}
[23:00:50] INFO: POST /api/v1/auth/login 200 194ms {"reqId":"a39e4a04-c2e6-43fa-8978-7d5beed15754"}
[23:00:50] INFO: POST /api/v1/auth/login 200 153ms {"reqId":"5448c045-a435-46ea-81cc-29f89d47480c"}
[23:00:50] INFO: Email sent {"reqId":"cae153c5-c2db-48f4-a895-794b2599bead","mail":"summary-ready","userId":"311d3bf0-dfea-4f74-a863-
9281b8092be1"}
[23:00:50] INFO: POST /api/v1/summaries 201 358ms {"reqId":"cae153c5-c2db-48f4-a895-794b2599bead"}
[23:04:44] WARN: POST /api/v1/recordings 403 106ms {"reqId":"dffa393b-1d8e-47f2-a263-7ab9158782a4"}
```

Evidencia 15 · Una línea por petición con su id. Con Mailpit apagado, el registro responde 201 y queda un `warn` `Email not sent` con el motivo

Logs en producción (JSON) sin contraseña, token ni cookie

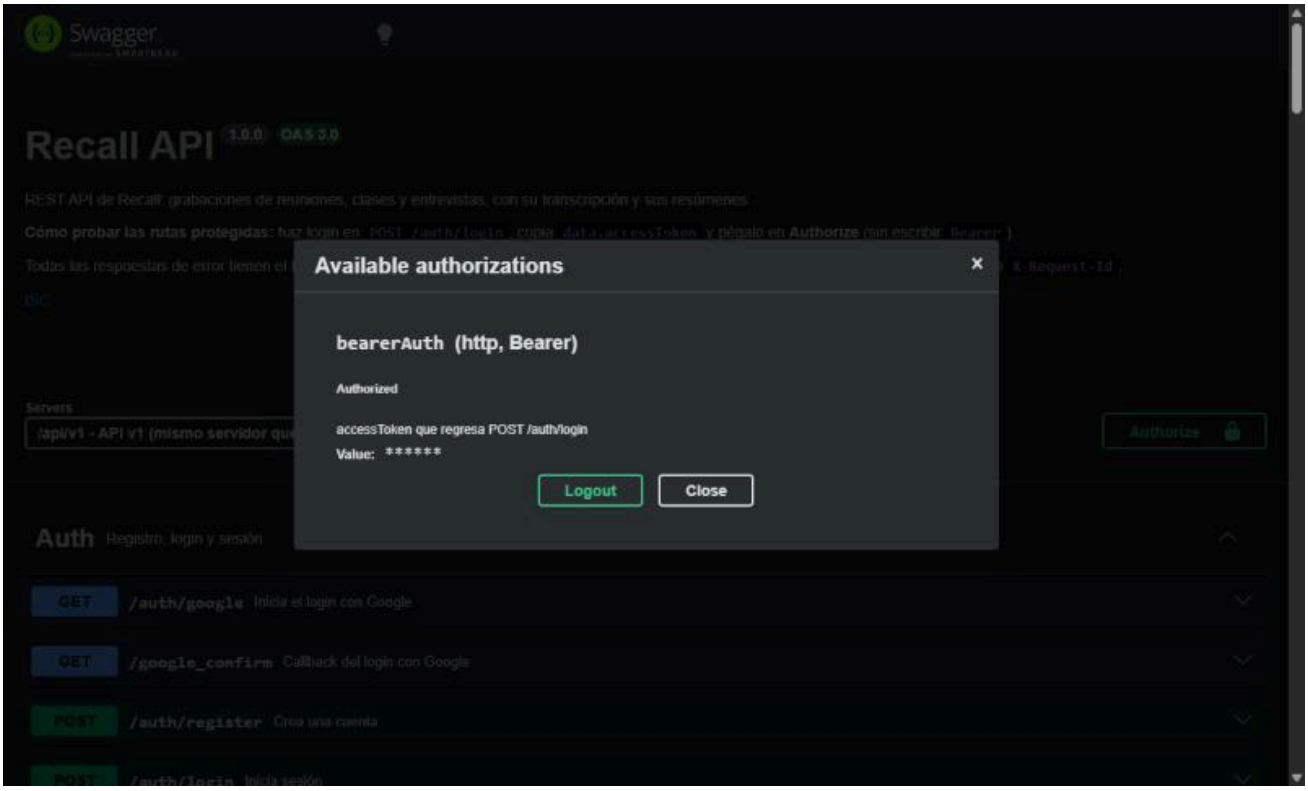
```
$ NODE_ENV=production npm start   (una línea JSON por evento)

{"level":"info","time":"2026-11-11T05:05:25.630Z","service":"recall-api","port":3989,"env":"production","logLevel":"info","msg":"Connected to
the database. Recall API listening"}
{"level":"info","time":"2026-11-11T05:05:26.563Z","service":"recall-api","reqId":"5a245467-55d1-4996-9a73-6c1cbaf3b63d","req":
{"method":"GET","url":"/health"},"res":{"statusCode":200},"responseTime":8,"msg":"GET /health 200 8ms"}
{"level":"warn","time":"2026-11-11T05:05:26.846Z","service":"recall-api","reqId":"demo-req-0001","email":"ana.lopez@recall.app","msg":"Login
failed"}
{"level":"warn","time":"2026-11-11T05:05:26.848Z","service":"recall-api","reqId":"demo-req-0001","req":
{"method":"POST","url":"/api/v1/auth/login"},"res":{"statusCode":401},"responseTime":202,"msg":"POST /api/v1/auth/login 401 202ms"}
{"level":"warn","time":"2026-11-11T05:05:26.932Z","service":"recall-api","reqId":"3e2a9e3c-bc00-4ef0-94c9-c9a663a5be58","req":
{"method":"GET","url":"/api/v1/users"},"res":{"statusCode":401},"responseTime":2,"msg":"GET /api/v1/users 401 2ms"}
```

Evidencia 16 · En producción cada evento es una línea JSON. El cliente mandó `X-Request-Id: demo-req-0001`, una contraseña, un token y una cookie: el id se respeta y ninguno de los secretos aparece

Además, se buscaron en las líneas del log de todas las pruebas las contraseñas usadas, el prefijo de los JWT (`eyJhbGci`), `Bearer`, `token=` y la cookie `recall_session=`: **ninguna aparece**.

Documentación interactiva



Evidencia 17 · `/docs` con la sesión autorizada (`bearerAuth`) y los endpoints agrupados por módulo

Requisitos del curso

Requisito	Cómo lo cubre Recall	Estado
Base de datos	PostgreSQL 17 con Prisma: 4 tablas relacionadas, índices únicos y migraciones versionadas.	Hecho
Validaciones	Middleware `validate` con zod para body, params y query; errores por campo con status 400.	Hecho
MVC	Modelos (Prisma + tipos), vistas (Handlebars) y controladores, conectados por rutas por módulo.	Hecho
Vistas	Handlebars: registro, login, inicio, listado y detalle de grabaciones, y usuarios, con datos reales y permisos.	Hecho

Requisito	Cómo lo cubre Recall	Estado
Autenticación y permisos	Registro y login propios con bcrypt y JWT; roles `USER` y `ADMIN` con matriz de permisos en el API y las vistas; correo verificado para subir audio.	Hecho
Login con proveedor	Google con OAuth 2.0 / OpenID Connect: vincula por correo verificado o crea un `USER`.	Hecho
Correos	nodemailer con Mailpit en desarrollo: verificación de correo, recuperación de contraseña y «Tu resumen está listo».	Hecho (esta entrega)
Archivos	Subida del audio con multer a un bucket privado de S3; la BD guarda solo `s3Key`.	Siguiente entrega
Servicio de terceros	Amazon Transcribe (transcripción) y Amazon Bedrock (resúmenes), además de Google y S3.	Google hecho; Transcribe y Bedrock pendientes
Tiempo real (websockets)	Socket.IO: el API emite el cambio de estado de una grabación (PROCESSING → DONE o FAILED) a la sala del usuario, para que la vista se actualice sin recargar.	Entrega posterior

Flujo de trabajo y CI/CD

- Ramas por funcionalidad (por ejemplo `auth_jwt`, `dev_emi_three`) que llegan a `main` por Pull Request revisado por el otro integrante. Nadie trabaja directo en `main`.
- El commit que se evalúa en cada entrega se etiqueta en `main`: `entrega-1`, `entrega-2` y `entrega-3`.
- En la entrega 3, cada requisito tuvo su issue (#9 a #18) y su rama, integrada por PR a `main` con `Closes #n`.

Workflow	Qué revisa
`ci.yml`	ESLint; `prisma validate`, typecheck y build; con un PostgreSQL y un Mailpit de servicio aplica las migraciones, corre el seed dos veces, arranca el API, corre la colección de Postman con newman y revisa que las vistas muestren datos con una sesión real.
`sonarqube.yml`	Análisis de SonarQube Cloud (bugs, code smells, duplicación y vulnerabilidades) con quality gate: si no pasa, el workflow falla.
`docker.yml`	Construye la imagen del API; en push a `develop` o `main` la publica en GitHub Container Registry.

Consideraciones

Seguridad y privacidad

- Las grabaciones contienen voces y datos personales, por lo que aplica la Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP).
- El usuario debe contar con el consentimiento de las personas grabadas.
- Las contraseñas se guardan con bcrypt y `passwordHash` nunca sale en una respuesta. El login no revela qué correos existen.
- `JWT_SECRET`, la contraseña del SMTP, las credenciales de Google y las de AWS solo viven en el `.env`, que está en `.gitignore`. El `.env.example` trae un `JWT_SECRET` de desarrollo que se rechaza en producción.
- `npm run seed` vacía las tablas: se usa solo en desarrollo.
- Los enlaces de los correos son de un solo uso, vencen y en la BD solo se guarda su sha256. Ni los tokens ni las contraseñas se escriben en el log.

Costos

- Google OAuth y Mailpit no tienen costo; para producción bastaría el plan gratuito de un servicio SMTP. S3, Transcribe y Bedrock cobran por uso: se configurará una alerta de AWS Budgets y una duración máxima de audio.
- GitHub Actions y SonarQube Cloud son gratuitos para este uso.

Próximos pasos

Entrega	Pendientes
Entrega 4: archivos	Subida de audios con multer a S3.
Entregas siguientes	Transcripción con Amazon Transcribe, resúmenes con Amazon Bedrock, websockets con Socket.IO, paginación y filtros. Como extras: límite de solicitudes en `forgot-password` y `resend-verification`, y un correo que avise que la contraseña cambió.